

AEKD – Gestion de Tontine

Rapport d'avancement du MVP — Backend (mise à jour)

Date du rapport	25 septembre 2026
Périmètre	Backend complet (§3 du CLAUDE.md) : auth, membres, séances, cotisations, paiements, fonds, sanctions, prêts, audit, dashboards, données de démo
Statut global	Backend fonctionnellement complet ; frontend Angular non démarré
Dépôt	https://github.com/takmalab/aekd-tontine-demo (branche master)
Tests automatisés	46/46 tests unitaires/intégration en succès, base de test isolée

Sommaire

- 1. Contexte et objectif
- 2. Fonctionnalités livrées, module par module
- 3. Modèle de données
- 4. Documentation interactive de l'API (Swagger)
- 5. Données de démonstration
- 6. Tests et isolation de la base de test
- 7. Décisions techniques prises (non métier)
- 8. Points métier encore à valider
- 9. Historique Git
- 10. Prochaines étapes

1. Contexte et objectif

Ce rapport fait le point sur l'ensemble des travaux réalisés sur le backend de l'application de gestion de la tontine AEKD depuis le rapport précédent (qui couvrait uniquement l'authentification, les séances et le début des cotisations). Le backend couvre désormais l'intégralité du périmètre fonctionnel du MVP tel que défini au CLAUDE.md §3.

Conformément à la méthode de développement progressif imposée par le cahier des charges (§35), chaque fonctionnalité a été livrée, compilée, testée et vérifiée manuellement en HTTP avant de passer à la suivante.

2. Fonctionnalités livrées, module par module

Module	Contenu
auth / user / member	Connexion (JWT), comptes utilisateurs, rôles (ADMIN/TRESORIER/MEMBRE), profils membres distincts des comptes.
session	Séances (périodes de gestion), créées par ADMIN/TRESORIER, consultables par tous.
contribution	Définition des cotisations (montant fixe ou libre, obligatoire/facultative, visibilité publique/privée, destination des fonds), participants, bénéficiaires, et rattachement à une séance via une « période » — modèle réutilisable d'un mois sur l'autre.
contribution (paiements)	Déclaration de paiement par le membre participant, validation/rejet réservés au trésorier, justificatifs (JPG/JPEG/PNG/PDF, stockage local abstrait).
fund	Solde du fonds commun et de l'épargne individuelle, calculés à partir des paiements validés (jamais mélangés).
sanction	Règles de sanction configurables (seuil de retard, montant ou description en nature) et application manuelle par le trésorier — aucune application automatique.
loan / repayment	Politiques de prêt configurables (ADMIN), demandes de prêt avec explication des règles respectées/non respectées, décision du trésorier, remboursements partiels avec calcul automatique du solde restant.
audit	Journal des actions sensibles (séances, cotisations, paiements, sanctions, prêts, remboursements), consultable par l'ADMIN uniquement.
dashboard	Tableau de bord unique dont le contenu s'adapte au rôle de l'appelant (vue globale pour ADMIN/TRESORIER, vue personnelle pour un membre).

Chaque endpoint applique les permissions par rôle telles que définies littéralement au §6 du CLAUDE.md. Deux nuances notables, vérifiées avec attention plutôt que supposées par analogie :

- Validation/rejet des paiements : réservé au TRESORIER — l'ADMIN peut seulement consulter.
- Configuration des règles de prêt : réservée à l'ADMIN — le TRESORIER ne l'a pas dans sa liste de permissions (contrairement aux sanctions, où les deux rôles peuvent configurer).
- Consultation du journal d'audit : réservée à l'ADMIN — le TRESORIER ne l'a pas non plus.

3. Modèle de données

10 migrations Flyway (V1 à V10) ont été appliquées, créant au total 18 tables. Le modèle suit celui du CLAUDE.md §27, avec un ajustement documenté :

- **ContributionDefinition** est un modèle réutilisable, indépendant d'une séance précise (au lieu d'un lien direct Session→Definition) ; c'est **ContributionPeriod** qui l'attache à une séance donnée. Ce choix reflète la nature récurrente/mensuelle des cotisations connues (§9) et le texte du §13, qui distingue explicitement définition et période.
- Aucune entité « Fund » séparée : les soldes (fonds commun, épargne individuelle) sont calculés à la volée à partir des paiements validés plutôt que stockés, pour ne jamais pouvoir dériver de la réalité des paiements.

Version	Contenu
V1 - V2	Rôles et utilisateurs
V3	Profils membres
V4	Séances
V5	Cotisations, participants, bénéficiaires, périodes
V6	Déclarations de paiement
V7	Justificatifs de paiement
V8	Règles de sanction et sanctions appliquées
V9	Politiques de prêt, prêts, évaluations de règles, remboursements
V10	Journal d'audit

4. Documentation interactive de l'API (Swagger)

L'intégration OpenAPI/Swagger prévue au §4 du CLAUDE.md (absente jusque-là) a été ajoutée : chaque endpoint est désormais documenté et testable directement depuis un navigateur, avec un bouton « Authorize » permettant de coller un jeton JWT pour appeler les routes protégées.

Accès une fois le backend démarré : <http://localhost:8080/swagger-ui/index.html>. Voir le guide d'utilisation séparé pour le détail des étapes.

5. Données de démonstration

Conformément au §32, un jeu de données de démonstration a été implémenté sous la forme d'un composant activé uniquement par le profil Spring demo (SPRING_PROFILES_ACTIVE=demo), jamais par défaut, et idempotent.

Il crée, en passant par la couche service réelle (donc avec audit automatique) :

- 1 administrateur (admin@aekd.cm) et 1 trésorier (tresorier@aekd.cm)
- 5 membres fictifs aux anciennetés variées (noms génériques, sans lien avec le propriétaire du projet)
- 1 séance correspondant au mois courant (toujours « active » quel que soit le jour d'exécution)
- 4 cotisations connues activées avec participants (50 000, 10 000, Ration, Banque/Épargne)
- 4 paiements (3 validés, 1 en attente)

- 1 règle de sanction et 1 politique de prêt (valeurs d'exemple du CLAUDE.md, explicitement documentées comme illustratives)
- 2 demandes de prêt (une en attente, une approuvée avec un remboursement partiel enregistré)

Note : la cotisation « Voir bébé » n'est volontairement pas créée : c'est la seule cotisation connue dont le montant ET la destination des fonds restent « à valider » (§9.4) — la créer aurait exigé d'inventer une réponse à l'une des deux questions en suspens.

6. Tests et isolation de la base de test

46 tests d'intégration (JUnit + Spring Boot Test) couvrent l'ensemble des modules, avec un accent particulier sur les règles de permission par rôle et les cas limites (montants invalides, doublons, dépassements de solde, auto-validation interdite, etc.).

Anomalie corrigée pendant cette session : les tests utilisaient initialement la même base de données que le serveur de développement. Une fois les données de démonstration seedées, plusieurs tests qui supposaient un état vierge (aucune séance, solde à zéro, aucune politique de prêt) ont commencé à échouer. Correction : une base `aekd_tontine_test` dédiée a été créée, et tous les tests utilisent désormais le profil `Spring test` pour s'y connecter, totalement isolés des données de développement/démonstration.

Résultat final : **Tests run: 46, Failures: 0, Errors: 0 – BUILD SUCCESS**, exécutés contre la base de test isolée.

7. Décisions techniques prises (non métier)

Choix purement architecturaux, sans impact sur les montants, seuils ou règles de fonctionnement de la tontine :

- Identifiants UUID pour toutes les entités.
- Modules « fund », « storage », « common », « config » ajoutés au-delà de la liste initiale du §5, pour des besoins d'infrastructure transverses non couverts par les modules métier listés.
- Détection des sanctions candidates : lecture seule, jamais d'application automatique (`docs/decisions.md` §10 reste « À VALIDER »).
- Le trésorier peut approuver un prêt même si une règle de politique n'est pas respectée (aucun rejet automatique inventé).
- Le solde d'un prêt se calcule contre le montant approuvé (principal) uniquement — le mode de calcul des intérêts reste « à définir » (§14) et n'est appliqué nulle part.

8. Points métier encore à valider

Le fichier `docs/decisions.md` recense toujours l'ensemble des décisions en attente (montants de « Voir bébé » et « Banque / Épargne », délais de retard, montants de sanctions, règles précises des prêts, etc.). Aucune n'a été tranchée arbitrairement : chaque fonctionnalité correspondante a été construite pour rester configurable sans valeur par défaut inventée.

9. Historique Git

Commit	Message
1b4ab38	feat: add demo data seeder (CLAUDE.md §32)
1db8f78	feat: add dashboards for admin/treasurer and members
fef9366	feat: add payment proofs, funds, sanctions, loans/repayments, and audit log
2fce917	feat: add Session, ContributionDefinition/Period, and payment declaration
b0f4e9f	feat: add User/Role/Member entities and auth login endpoint

6f452b3	feat: initialize git repo and make backend startable
---------	--

L'ajout de Swagger/OpenAPI et la correction de l'isolation des tests (section 6) sont en cours au moment de la rédaction de ce rapport et n'apparaissent pas encore dans cet historique.

10. Prochaines étapes

- Commit/push des derniers ajustements (Swagger, isolation des tests).
- Démarrage du frontend Angular (Phase 4 du §35) : structure recommandée au §29, en s'appuyant sur l'API désormais complète et documentée.
- Intégration frontend/backend (Phase 5), puis tests de bout en bout (Phase 6).